

Topic – Inheritance & super

A. Inheritance — 30 Interview Questions

1. What exactly happens when Python looks up an attribute on an object whose class uses inheritance?
2. Is inheritance in Python based on classes, objects, or both? Explain precisely.
3. If a child class does not override a method, how does Python locate that method?
4. If both parent and child define an attribute with the same name, which one is accessed through the child object, and why?
5. Can a child class access a parent's `__private` attribute directly? Why or why not?
6. What is name mangling, and does it actually make a Python attribute private?
7. If a parent class has a class variable and the child modifies it using `self.variable`, does the parent class variable change?
8. What is the difference between modifying an inherited class attribute and creating an instance attribute with the same name?
9. If a parent has `__init__()` and the child defines its own `__init__()`, is the parent's constructor automatically executed?
10. Why does Python allow a subclass to completely change the behavior of an inherited method?
11. Can a subclass override a `@staticmethod` with an instance method? What happens?
12. Can a subclass override a `@classmethod` with a normal instance method? What changes?
13. Can a child class inherit from multiple classes in Python? If yes, how does Python decide which implementation to use?
14. What is MRO, and why is it necessary in Python's multiple inheritance?
15. How is Python's MRO different from simply searching the first parent, then its parent?
16. What is the MRO of this class hierarchy?
17. `class A: pass`
18. `class B(A): pass`
19. `class C(A): pass`
`class D(B, C): pass`
20. What problem does the C3 linearization algorithm solve?
21. Can Python create an MRO for every possible multiple-inheritance hierarchy? If not, when does it fail?
22. What is the diamond problem, and how does Python resolve it?
23. If two parent classes define the same method, which method does the child execute?
24. Does changing the order of parent classes change the behavior of a class with multiple inheritance?
25. Why is this valid?
26. `class C(A, B):`
`pass`
but changing it to:
`class C(B, A):`
`pass`
can produce different behavior?
27. Can a class inherit from two classes that themselves have a common parent?
28. What happens if two parent classes have incompatible inheritance relationships?
29. Can you override an inherited property with a normal attribute? What happens during attribute lookup?

30. Can you override a normal method with a property? What changes for the caller?
 31. Is method overriding checked by Python at compile time or determined at runtime?
 32. Why does Python support runtime polymorphism without requiring explicit interfaces?
 33. When does inheritance create tighter coupling than composition?
 34. If a class inherits from another class but uses only one or two inherited features, what design principle might indicate that inheritance is inappropriate?
-

B. super() — Interview Questions

1. What exactly does super() return?
2. Is super() a function, an object, or a keyword?
3. Why is saying “**super() calls the parent class**” technically incorrect?
4. What does super() use to determine where method lookup should begin?
5. What is the difference between:
super().method()
and:
Parent.method(self)
6. Why can Parent.method(self) break a multiple-inheritance design while super().method() can preserve it?
7. Does super() always call the immediate parent?
8. Can super() call a method from a class that is not the direct parent?
9. What role does MRO play when super() is used?
10. Consider:

```
class A:  
def show(self):  
print("A")  
  
class B(A):  
def show(self):  
print("B")  
super().show()
```

What exactly is super() searching?

11. Why can super() work even when the class doesn't explicitly mention its parent?
12. What is the difference between:

```
super()  
  
and:  
  
super(B, self)
```

13. Why is zero-argument `super()` possible in Python 3?
14. What hidden information does Python use when executing zero-argument `super()`?
15. Can `super()` be used outside a class method?
16. Why can calling `super()` inside a nested function behave differently from calling it directly inside a method?
17. What happens if a child overrides a method but does not call `super()`?
18. Is calling `super()` mandatory whenever a method is overridden?
19. Why is `super()` particularly important in cooperative multiple inheritance?
20. Consider:

```
class A:  
    def show(self):  
        print("A")  
  
class B(A):  
    def show(self):  
        print("B")  
        super().show()  
  
class C(A):  
    def show(self):  
        print("C")  
        super().show()  
  
class D(B, C):  
    pass
```

What is the exact output of:

D().show()

21. Why does `C.show()` execute in the previous example even though C is not the direct parent of D?
22. What would change if `B.show()` used:

A.show(self)

instead of:

super().show()

23. What happens if one class in a cooperative inheritance chain does not call `super()`?
24. Can `super()` skip a class in the MRO?
25. Can `super()` call a method that exists only several levels above the current class?
26. Why can `super()` behave unexpectedly if the inheritance hierarchy is changed later?
27. What is cooperative inheritance, and why does it require all participating classes to cooperate?

28. Why are `*args` and `**kwargs` commonly used with `super().__init__()` in multiple inheritance?
29. If every class in a multiple-inheritance chain calls `super()`, how does Python prevent the common ancestor from being initialized multiple times?
30. Final cross-question: If `super()` follows MRO rather than simply calling the parent, then what exactly determines the next method that `super()` will execute?